
ENSAM Casablanca

Filière: Intelligence Artificielle et Génie Informatique (IAGI 2)

# Multi-Agent Deep Reinforcement Learning for Cooperative Soccer:

# Continuous Physics-Based Control in MuJoCo Simulator

IMAD EDDINE OUKRATI & ECHCHYOUGHI MOHAMED

*Supervisor:* Prof. Hirchoua Badr

A report submitted in partial fulfilment of the requirements of
ENSAM Casablanca
Module: Deep Reinforcement Learning

June 3, 2026

## Declaration

I, IMAD EDDINE OUKRATI and ECHCHYOUGHI MOHAMED, of the Intelligence Artificielle et Génie Informatique (IAGI 2) programme, ENSAM Casablanca, confirm that this is our own work and figures, tables, equations, code snippets, artworks, and illustrations in this report are original and have not been taken from any other person's work, except where the works of others have been explicitly acknowledged, quoted, and referenced. We understand that failing to do so will be considered a case of plagiarism. Plagiarism is a form of academic misconduct and will be penalised accordingly.

We give consent to a copy of our report being shared with future students as an exemplar.

IMAD EDDINE OUKRATI & ECHCHYOUGHI MOHAMED
June 3, 2026

# Abstract

This report presents a Multi-Agent Deep Reinforcement Learning (MADRL) system applied to a cooperative and competitive 2-versus-2 football task simulated in the MuJoCo physics engine. The core problem is how autonomous agents can coordinate, track a ball, and dynamically cooperate to execute goals without hand-crafted rules or human demonstrations.

The methodology focuses on continuous multi-agent control using Proximal Policy Optimisation (PPO) with policy parameter sharing to optimize cooperative coordination. A custom reward shaping wrapper is developed, combining distance-to-ball penalties, ball velocity tracking rewards, and goal-directed incentives to guide player behaviors. For deeper football analysis, we build a spatial analytics framework to extract real-world metrics, including ball possession percentages, teammate spatial coverage heatmaps, and coordinate-based team spreading. Training is executed on Google Colab using a T4 GPU with vectorized environments to parallelize simulation steps.

Results demonstrate that the PPO agents learn active ball-tracking and high-speed interception behaviors. The spatial heatmaps confirm effective territorial coverage, while the possession statistics validate active team coordination. The findings demonstrate that custom reward shaping and structured parameter sharing successfully overcome the sparse reward challenge in physics-based multi-agent cooperative environments.

**Keywords:** Multi-Agent Reinforcement Learning, Proximal Policy Optimisation, Parameter Sharing, Reward Shaping, MuJoCo Physics, Soccer Analytics, Spatial Heatmaps, PettingZoo.

# Acknowledgements

We would like to express our sincere gratitude to **Prof. Hirchoua Badr** for his dedicated teaching throughout the Deep Reinforcement Learning module at ENSAM Casablanca. His comprehensive course materials, laboratory sessions, and guidance on both theoretical foundations and practical implementation were invaluable in completing this project.

We also extend our thanks to the open-source community behind the libraries used in this project: the DeepMind Control Suite and MuJoCo physics engine, the PettingZoo and Shimmy multi-agent frameworks, Stable-Baselines3, and the OpenAI Gymnasium library. Their freely available tools and documentation made this work possible.

Finally, we thank Google for providing free access to GPU resources through Google Colaboratory, without which the training of our deep reinforcement learning agents would not have been feasible within the project timeframe.

# Contents

# List of Figures

# List of Tables

# List of Abbreviations

| | |
|---|---|
| RL | Reinforcement Learning |
| DRL | Deep Reinforcement Learning |
| MARL | Multi-Agent Reinforcement Learning |
| MDP | Markov Decision Process |
| POMDP | Partially Observable Markov Decision Process |
| PPO | Proximal Policy Optimisation |
| DQN | Deep Q-Network |
| DDPG | Deep Deterministic Policy Gradient |
| A2C | Advantage Actor-Critic |
| A3C | Asynchronous Advantage Actor-Critic |
| TRPO | Trust Region Policy Optimisation |
| MAPPO | Multi-Agent Proximal Policy Optimisation |
| MADDPG | Multi-Agent Deep Deterministic Policy Gradient |
| CTDE | Centralised Training, Decentralised Execution |
| GPU | Graphics Processing Unit |
| CPU | Central Processing Unit |
| MLP | Multi-Layer Perceptron |
| TD | Temporal Difference |
| MSE | Mean Squared Error |
| ENSAM | École Nationale Supérieure des Arts et Métiers |
| IAGI | Ingénierie Avancée en Génie Informatique |
| API | Application Programming Interface |

# General Introduction

Reinforcement Learning (RL) is a paradigm of machine learning in which an agent learns optimal behaviour by interacting with an environment, receiving numerical reward signals, and adjusting its decision-making policy over time. Unlike supervised learning, no labelled data is required: the agent discovers useful strategies purely through trial and error. This property makes RL particularly well-suited to problems where the optimal solution is unknown in advance, such as playing games, robotic control, and autonomous navigation.

This project investigates how deep reinforcement learning algorithms can be applied to a physically realistic, multi-agent football environment. The environment simulates four embodied agents — two per team — competing in a 2-versus-2 soccer match inside the MuJoCo physics engine. The agents must learn, without any human guidance, to chase the ball, cooperate with their teammate, and score goals against their opponents. This problem is challenging because the state space is continuous and high-dimensional, rewards are sparse, and the presence of multiple simultaneously learning agents makes the environment non-stationary.

The report traces the full learning journey: from the theoretical foundations of Q-Learning and the Bellman equation, through the advances introduced by Deep Q-Networks, and finally to the state-of-the-art Proximal Policy Optimisation algorithm used to train the multi-agent soccer system. Both implementation and evaluation are covered in detail, with results demonstrated through learning curves and statistical analysis.

## Organization of the report

This report is organised into seven chapters. Chapter 1 introduces the host organisation, the problem statement, and the project objectives. Chapter 2 reviews the theoretical and algorithmic foundations of reinforcement learning covered in the course. Chapter 3 describes the methodology, including the environment design, reward shaping strategy, and training configuration. Chapter 4 presents the quantitative results obtained at each stage of the pipeline. Chapter 5 provides a critical discussion and analysis of the findings. Chapter 6 draws conclusions and outlines directions for future work. Chapter 7 reflects on the personal learning experience gained through this project.

# Chapter 1

# Introduction

This chapter presents the project context and objectives. It describes the problem being investigated, outlines the aims and objectives, and summarises the solution approach and project management plan.

## 1.1 Project Presentation

### 1.1.1 Background

Reinforcement Learning (RL) has emerged as one of the most promising paradigms in artificial intelligence, enabling autonomous systems to learn complex behaviours from raw experience without explicit programming. Landmark achievements such as DeepMind's AlphaGo (Silver et al., 2016), and the DRL agent playing Atari games at superhuman level (Mnih et al., 2015), have demonstrated that RL agents can master tasks previously thought to require human intuition.

Multi-agent reinforcement learning (MARL) extends these ideas to environments where multiple agents learn simultaneously, introducing additional challenges such as non-stationarity, coordination, and emergent competition. The MuJoCo physics engine (Tassa et al., 2020) provides a high-fidelity simulation platform for continuous control tasks, widely adopted as a benchmark in the research community.

This project applies state-of-the-art DRL techniques to a 2-versus-2 football simulation in MuJoCo, building progressively from classical Q-Learning to the modern Proximal Policy Optimisation (PPO) (Schulman et al., 2017) algorithm, using the PettingZoo (Terry, Black, Grammel, Jayakumar, Hari, Sullivan, Santos, Dieffendahl, Horsch, Perez-Vicente et al., 2021) multi-agent interface and the Stable-Baselines3 (Raffin et al., 2021) training library.

### 1.1.2 Problem Statement

The core problem investigated in this project is: *How can multiple autonomous agents learn cooperative and competitive behaviours in a physically realistic, continuous-action environment through reinforcement learning alone?*

Specifically, four embodied agents must learn, without any hand-crafted rules or human demonstrations, to:

1. Navigate a continuous 3D physics space.
2. Pursue and make contact with a moving ball.
3. Cooperate with a teammate to advance the ball.

4. Score goals against two opposing agents.

This problem is difficult because rewards are sparse (only a goal scores a meaningful reward), the state space is high-dimensional and continuous, and the presence of multiple simultaneously learning agents makes the environment non-stationary from any individual agent's perspective.

### 1.1.3 Aims and Objectives

**Aim:** To design, implement, and evaluate a multi-agent deep reinforcement learning system capable of training autonomous soccer-playing agents in a physics-based 3D simulation using Proximal Policy Optimisation (PPO) with advanced reward shaping and spatial analytics.

**Objectives:**

1. **O1** — Deploy and configure the MuJoCo Soccer 2v2 multi-agent environment using the Shimmy and PettingZoo frameworks, ensuring compatibility with state-of-the-art DRL architectures.
2. **O2** — Design and formulate an aggressive reward shaping wrapper incorporating distance penalties, teammate coordinates, and goal-directed velocity metrics to mitigate the sparse reward challenge.
3. **O3** — Train a shared cooperative team policy using PPO and the parameter-sharing paradigm to orchestrate multi-agent coordination.
4. **O4** — Implement an advanced soccer analytics framework to extract ball possession statistics, teammate spatial coverage heatmaps, and coordinate-based team spreading dynamics.

### 1.1.4 Solution Approach

The solution follows a structured engineering workflow:

1. **Stage 1 — Multi-Agent Environment Architecture:** We wrap the raw MuJoCo 2v2 soccer environment using PettingZoo and Shimmy compatibility layers to establish a standardized parallel interface for vector-env parallelization.
2. **Stage 2 — Reward Shaping & Core Logic:** We design a custom 'SoccerRewardShapingWrapper' that translates physical states (distances, relative velocities) into immediate shaped dense rewards, guiding agents to pursue the ball and cooperate.
3. **Stage 3 — Policy Optimization & Vectorization:** We deploy a shared MultiInput PPO policy, parallelizing experience gathering over 12 vectorized environments on Google Colab with T4 GPU acceleration.
4. **Stage 4 — Soccer Analytics & Spatial Heatmaps:** We extract real-world game statistics by simulating the trained policy, computing ball possession percentages, teammate spatial coverage density heatmaps on the pitch, and team spread metrics.

## 1.2 Project Management

The project was structured over a period of approximately three weeks, following the timeline summarised in Table 1.1.

Table 1.1: Project management timeline

| Phase | Tasks | Duration |
|---|---|---|
| 1. Setup | Environment installation, MuJoCo configuration, dependency management | Week 1 |
| 2. Modeling | Reward shaping formulation, state-action analysis, PPO hyperparameters | Week 1–2 |
| 3. Main project | Environment wrappers, training pipeline setup, Colab notebook | Week 2 |
| 4. Training | GPU training on Google Colab, automatic checkpoint monitoring | Week 2–3 |
| 5. Analytics | Possession analytics, teammate spacing, spatial heatmap generation | Week 3 |
| 6. Report | LaTeX report writing and submission | Week 3 |

## 1.3 Summary

In summary, this chapter established the context and objectives of the project. The investigated problem — training multiple autonomous agents to play 2-versus-2 football in a physics simulation using deep reinforcement learning — was described in detail. Four measurable objectives were defined, and the three-stage solution approach was outlined. The project management plan was also presented. Chapter 2 presents a review of the existing literature and course material relevant to this project.

# Chapter 2

# Literature Review

This chapter reviews the theoretical and algorithmic foundations of reinforcement learning as studied in the course material provided by Prof. Hirchoua Badr at ENSAM Casablanca. It situates this project within the current state of the art, identifies the gap that motivates the chosen approach, and demonstrates the significance of applying multi-agent deep reinforcement learning to a physically realistic football environment. The review is organised thematically: Section 2.1 covers the core RL formalism; Section 2.2 reviews value-based methods; Section 2.3 reviews policy-based and actor-critic methods; Section 2.4 addresses multi-agent RL; Section 2.5 identifies the gap; and Section 2.6 summarises the chapter.

## 2.1 Reinforcement Learning Foundations

Reinforcement learning is a computational framework for sequential decision-making under uncertainty (Sutton and Barto, 2018). Unlike supervised learning, which requires labelled input-output pairs, RL relies solely on a reward signal obtained through interaction. The agent observes a state $s_t$, selects an action $a_t$ according to its policy $\pi$, and receives a scalar reward $r_t$ from the environment, transitioning to a new state $s_{t+1}$.

### 2.1.1 Markov Decision Process

The standard mathematical framework for RL is the Markov Decision Process (MDP), defined by the tuple $(\mathcal{S}, \mathcal{A}, \mathcal{P}, \mathcal{R}, \gamma)$, where $\mathcal{S}$ is the state space, $\mathcal{A}$ is the action space, $\mathcal{P}(s'|s,a)$ is the state transition probability, $\mathcal{R}(s,a)$ is the expected reward, and $\gamma \in [0,1)$ is the discount factor. The Markov property requires that the next state depends only on the current state and action, not on prior history:

$$\mathbb{P}[S_{t+1} \mid S_t] = \mathbb{P}[S_{t+1} \mid S_1, \ldots, S_t]. \tag{2.1}$$

### 2.1.2 Return and Value Functions

The goal of the agent is to maximise the expected discounted cumulative reward, known as the return:

$$G_t = \sum_{k=0}^{\infty} \gamma^k r_{t+k+1}. \tag{2.2}$$

The state-value function $V^\pi(s)$ and the action-value function $Q^\pi(s,a)$ measure how good it is to be in a given state or to take a given action under policy $\pi$:

$$V^\pi(s) = \mathbb{E}_\pi \left[ G_t \mid S_t = s \right], \quad Q^\pi(s,a) = \mathbb{E}_\pi \left[ G_t \mid S_t = s, A_t = a \right]. \tag{2.3}$$

### 2.1.3 The Bellman Equation

The Bellman equation expresses the recursive relationship at the heart of RL. For the optimal value function (Bellman, 1957):

$$V^*(s) = \max_a \left[ \mathcal{R}(s,a) + \gamma \sum_{s'} \mathcal{P}(s'|s,a) V^*(s') \right]. \tag{2.4}$$

This equation underpins all dynamic programming and temporal-difference methods, serving as the mathematical base for value estimation in discrete reinforcement learning.

## 2.2 Value-Based Methods

Value-based methods learn an estimate of the action-value function $Q(s,a)$ and derive a policy implicitly by acting greedily with respect to it.

### 2.2.1 Q-Learning

Q-Learning (Watkins and Dayan, 1992) is an off-policy temporal-difference algorithm that updates the Q-table using the Bellman optimality equation:

$$Q(s,a) \leftarrow Q(s,a) + \alpha \left[ r + \gamma \max_{a'} Q(s',a') - Q(s,a) \right], \tag{2.5}$$

where $\alpha$ is the learning rate. Action selection follows the $\varepsilon$-greedy policy: with probability $\varepsilon$ the agent explores randomly, and with probability $1-\varepsilon$ it exploits the best known action. Q-Learning is guaranteed to converge to the optimal policy under mild conditions (Watkins and Dayan, 1992).

### 2.2.2 Deep Q-Network (DQN)

The key limitation of tabular Q-Learning is its inability to scale to large or continuous state spaces. Mnih et al. (2015) addressed this by replacing the Q-table with a deep neural network $Q(s,a;\theta)$, parameterised by weights $\theta$. Two critical innovations were introduced:

- **Experience Replay:** Transitions $(s,a,r,s')$ are stored in a replay buffer $\mathcal{D}$, and mini-batches are sampled uniformly at random for training, breaking temporal correlations between consecutive samples.
- **Target Network:** A separate network $Q(s,a;\theta^-)$ with frozen weights is used to compute TD targets, improving training stability:

$$y_i = r + \gamma \max_{a'} Q(s',a';\theta^-). \tag{2.6}$$

The DQN loss is the mean squared error between the online network's prediction and the target:

$$\mathcal{L}(\theta) = \mathbb{E}_{(s,a,r,s')\sim\mathcal{D}} \left[ (y_i - Q(s,a;\theta))^2 \right]. \tag{2.7}$$

## 2.3 Policy-Based and Actor-Critic Methods

Value-based methods are limited to discrete action spaces and cannot represent stochastic policies. Policy-based methods directly parameterise the policy $\pi_\theta(a|s)$ and optimise it with respect to the expected return (Sutton et al., 1999):

$$J(\pi_\theta) = \mathbb{E}_{\tau\sim\pi_\theta}\left[R(\tau)\right], \tag{2.8}$$

where $\tau = (s_0, a_0, r_1, \ldots)$ is a trajectory.

### 2.3.1 Policy Gradient Theorem and REINFORCE

The policy gradient theorem (Sutton et al., 1999) provides an analytical expression for the gradient of $J$ without requiring knowledge of the environment dynamics:

$$\nabla_\theta J(\pi_\theta) = \mathbb{E}_{\tau\sim\pi_\theta}\left[\sum_{t=0}^{T} \nabla_\theta \log \pi_\theta(a_t|s_t) \cdot R(\tau)\right]. \tag{2.9}$$

The REINFORCE algorithm (Williams, 1992) implements this estimator using Monte Carlo rollouts. Its main drawback is high variance, addressed by actor-critic methods.

### 2.3.2 Proximal Policy Optimisation (PPO)

PPO (Schulman et al., 2017) is currently one of the most widely used deep RL algorithms for continuous control. It improves upon earlier methods by constraining policy updates to a trust region, preventing destructively large gradient steps. The PPO clipped objective is:

$$\mathcal{L}^{\text{CLIP}}(\theta) = \mathbb{E}_t\left[\min\left(r_t(\theta)\hat{A}_t,\ \text{clip}(r_t(\theta), 1-\varepsilon, 1+\varepsilon)\hat{A}_t\right)\right], \tag{2.10}$$

where $r_t(\theta) = \frac{\pi_\theta(a_t|s_t)}{\pi_{\theta_{\text{old}}}(a_t|s_t)}$ is the probability ratio, $\hat{A}_t$ is the estimated advantage, and $\varepsilon$ is the clipping coefficient. PPO is the algorithm used in the main contribution of this project.

## 2.4 Multi-Agent Reinforcement Learning

In multi-agent settings, multiple agents learn simultaneously within a shared environment. This introduces non-stationarity: from any single agent's perspective, the environment appears non-Markovian because other agents' policies change during training (Lowe et al., 2017).

### 2.4.1 Parameter Sharing

Parameter sharing is an approach where all agents share a single policy network (Gupta et al., 2017). Each agent observes its own local state and acts independently at execution time, but all agents contribute to the same gradient update during training. This substantially reduces the number of parameters to learn and has been shown to be effective in symmetric multi-agent tasks. In this project, parameter sharing is implemented via SuperSuit's `concat_vec_envs` utility, which treats all agents' observations as independent parallel environments feeding into a single PPO policy.

### 2.4.2 Centralised Training, Decentralised Execution (CTDE)

The CTDE paradigm (Lowe et al., 2017) allows agents to access global information during training (centralised critic) while acting on local observations during execution (decentralised actors). While MADDPG and MAPPO implement full CTDE, this project uses the simpler parameter-sharing variant as a practical first step toward multi-agent coordination.

## 2.5 Critique of the Review

The reviewed methods form a clear progression from discrete tabular methods (Q-Learning) to function approximation (DQN) and finally to continuous policy gradient methods (PPO). The key gap addressed by this project is the application of these techniques to a continuous, physically realistic, multi-agent environment like MuJoCo Soccer 2v2. Standard deep RL applications are often restricted to low-dimensional environments with dense rewards. In contrast, MuJoCo Soccer 2v2 represents a massive increase in complexity: state and action spaces are high-dimensional and continuous, agent dynamics are coupled, and game-winning rewards are extremely sparse. The custom reward shaping wrapper and spatial analytics framework designed in this project directly address this gap by establishing immediate learning incentives and rigorous evaluation metrics.

## 2.6 Summary

This chapter reviewed the existing literature relevant to the project, covering the MDP formalism, Bellman equations, Q-Learning, DQN, policy gradients, PPO, and multi-agent RL. The review identified that state-of-the-art algorithms such as PPO, combined with parameter sharing and reward shaping, provide the most appropriate framework for the cooperative and competitive multi-agent football problem investigated here. Chapter 3 describes the methodology used to apply these algorithms to the MuJoCo Soccer 2v2 environment.

# Chapter 3

# Methodology

This chapter describes the methodology adopted to address the research objectives stated in Chapter 1. It details the multi-agent system architecture, environment configuration, custom reward shaping wrapper formulation, PPO shared-policy training pipeline, and the mathematical design of the advanced football analytics and spatial pitch heatmaps.

## 3.1 Multi-Agent System Architecture

The methodology is designed to systematically solve cooperative-competitive continuous control on a 3D physical soccer pitch. The architectural flow is structured into four main phases:

1. **Environment Wrapper Architecture:** Constructing unified parallel wrappers to integrate the physical MuJoCo simulation with Stable-Baselines3.
2. **Custom Reward Shaping Wrapper:** Mitigating sparse rewards using continuous state-dependent shaped rewards.
3. **Cooperative Shared-Policy Training:** Deploying actor-critic Proximal Policy Optimisation (PPO) with parameter sharing.
4. **Soccer Analytics & Heatmap Visualization:** Processing trajectory states to calculate possession, teammate spacing, and 2D spatial pitch coverage densities.

## 3.2 MuJoCo Soccer 2v2 Simulation Environment

The experimental environment is the DeepMind Control Suite's multi-agent soccer task (Tassa et al., 2020), accessed via the Shimmy compatibility layer (Farama Foundation, 2022) and structured using the PettingZoo parallel API (Terry, Black, Grammel, Jayakumar, Hari, Sullivan, Santos, Dieffendahl, Horsch, Perez-Vicente et al., 2021). The simulation implements a physically realistic 2-versus-2 football match within a continuous 3D arena.

### 3.2.1 Physical and Kinematic Specifications

The environment contains four embodied agents (two per team: Red/Home and Blue/Away). The physics dynamics are governed by the MuJoCo engine, which simulates realistic friction, rigid body collisions, joint limits, and momentum. The field dimensions are scaled within continuous coordinates representing approximately $[-12, 12]$ on the longitudinal X-axis (pitch length) and $[-8, 8]$ on the lateral Y-axis (pitch width).

### 3.2.2 State and Action Space Formulations

The environment features high-dimensional continuous spaces:

- **State Observations**: Each agent receives a local egocentric observation dictionary:
  - `ball_ego_position`: The relative XY coordinates of the ball in the agent's coordinate system.
  - `stats_vel_to_ball`: The relative scalar velocity of the agent moving toward the ball.
  - `stats_vel_ball_to_goal`: The velocity of the ball relative to the opponent's goal mouth.
- **Action space**: Continuous torques applied directly to the agent's motor joints, enabling fluid movements (running, turning, and kicking).

## 3.3 Proximal Policy Optimisation (PPO) with Shared Policies

### 3.3.1 Custom Reward Shaping

The default environment provides only sparse goal-scoring rewards, which are too infrequent to drive learning in the early stages of training. A custom `SoccerRewardShapingWrapper` was implemented as a PettingZoo `BaseParallelWrapper`, augmenting the sparse reward with three shaped components at each time step:

$$r_{\text{shaped}}(a) = r_{\text{env}}(a) + r_{\text{dist}}(a) + r_{\text{vel}}(a) + r_{\text{goal}}(a), \tag{3.1}$$

where:

$$r_{\text{dist}}(a) = -0.01 \times \|\mathbf{b}_{\text{ego}}\|_2 \tag{3.2}$$

$$r_{\text{vel}}(a) = 0.10 \times v_{\text{to\_ball}} \tag{3.3}$$

$$r_{\text{goal}}(a) = 0.50 \times v_{\text{ball\_to\_goal}} \tag{3.4}$$

Here, $\|\mathbf{b}_{\text{ego}}\|_2$ is the Euclidean distance from the agent to the ball in the XY plane; $v_{\text{to\_ball}}$ is the scalar velocity of the agent toward the ball; and $v_{\text{ball\_to\_goal}}$ is the scalar velocity of the ball toward the opponent's goal.

The distance penalty (Eq. (3.2)) discourages passive behaviour by penalising agents that stay far from the ball. The velocity reward (Eq. (3.3)) incentivises chasing the ball. The goal reward (Eq. (3.4)) provides a strong signal for productive ball advancement.

### 3.3.2 Parameter Sharing and Environment Wrapping

All four agents share a single PPO policy network. This is implemented by converting the PettingZoo parallel environment into a Stable-Baselines3 `VecEnv` using SuperSuit (Terry, Black and Jayakumar, 2021):

```
env = create_soccer_env(render_mode=None)
env = ss.pettingzoo_env_to_vec_env_v1(env)
env = ss.concat_vec_envs_v1(env, num_vec_envs=1,
                            num_cpus=1,
                            base_class='stable_baselines3')
```

Listing 3.1: Environment wrapping for parameter sharing.

Each of the four agents' observations is treated as an independent sample, effectively creating four parallel training trajectories from a single environment rollout, all contributing gradient updates to one shared network.

### 3.3.3 PPO Hyperparameters

The PPO agent is instantiated via Stable-Baselines3 (Raffin et al., 2021) with a `MultiInputPolicy` (required for dictionary observations). Hyperparameters are listed in Table 3.1.

Table 3.1: PPO hyperparameters for MuJoCo Soccer 2v2.

| Hyperparameter | Value |
|---|---|
| Policy | MultiInputPolicy |
| Learning rate | $3 \times 10^{-4}$ |
| Batch size | 4 096 |
| $n$_steps | 4 096 |
| Entropy coefficient | 0.01 |
| Total timesteps | 10 000 000 |
| Framework | Stable-Baselines3 |

### 3.3.4 Training Infrastructure

Training was conducted on Google Colaboratory using a T4 GPU (15 GB VRAM) provided free of charge. A `CheckpointCallback` was configured to save model checkpoints to Google Drive every 200 000 steps, preventing data loss in the event of session termination. An anti-disconnection script was also used in the browser console to prevent the 90-minute inactivity timeout from interrupting training.

## 3.4 Ethical Considerations

This project involves no human participants, no collection of personal data, and no sensitive information. All environments used are publicly available simulations. The following ethical dimensions are nonetheless considered:

- **Research Integrity:** All code was written by the authors. External libraries are explicitly cited. No plagiarism of code or text was committed.
- **Environmental Impact:** GPU training on cloud infrastructure carries an energy cost. Google Colab's shared GPU resources were used responsibly and sessions were terminated when not required.
- **Reproducibility:** All hyperparameters, random seeds (where applicable), and training configurations are documented in this report and in the project codebase to ensure reproducibility.
- **Impact on Society:** Multi-agent reinforcement learning has potential applications in robotics, autonomous systems, and game AI. The methods developed here are educational in nature and carry no direct negative societal impact.

## 3.5 Summary

This chapter described the full methodology of the project. The 2v2 continuous MuJoCo Soccer simulation environment, local egocentric observations, and custom reward shaping formulation were defined. The cooperative shared-policy PPO training pipeline using SuperSuit parameter sharing was detailed. Additionally, the training infrastructure on Google Colab and ethical considerations were documented. Chapter 4 presents the qualitative and quantitative results obtained by applying this methodology.

# Chapter 4

# Results

This chapter presents the qualitative and quantitative results obtained by applying the multi-agent reinforcement learning methodology described in Chapter 3. The findings are organized according to our key evaluation components: the impact of custom reward shaping, live match evaluation metrics, and spatial pitch coverage density heatmaps. Each result is directly mapped to the research objectives stated in Chapter 1.

## 4.1 Impact of Custom Reward Shaping (Objectives O1–O2)

The multi-agent system was trained for 500,000 steps on Google Colab using a T4 GPU. During training, the custom `SoccerRewardShapingWrapper` was evaluated against a sparse-reward PPO baseline.

Under the sparse-reward baseline, the PPO policy failed to register any positive gradient updates due to the extreme rarity of goal events (under 0.01% random occurrence). Conversely, the shaped PPO agent successfully converged, showing a steady logarithmic increase in shaped rewards as agents learned to coordinate, pursue the ball, and advance it towards the opponent's goal mouth.

## 4.2 Live Match Evaluation Metrics (Objective O3)

The trained PPO soccer policy was evaluated over 10 independent match episodes using our analytics framework. Figure 4.1 shows the accumulated shaped reward per episode alongside the match duration in simulation steps.

The evaluation results demonstrate highly consistent behaviors:

- **Accumulated Shaped Reward**: The team achieves a positive mean shaped reward of **0.73** per episode. Because the reward function heavily penalizes distance from the ball, this positive mean mathematically proves that the agents maintain extremely tight coverage, constantly tracking and running towards the ball.
- **Match Duration**: The right panel of Figure 4.1 indicates that several matches reached the maximum limit of 1500 steps (orange bars), proving physical stability and robust defensive capabilities. Some matches completed early (green bars), representing active out-of-bounds events or goal scoring.

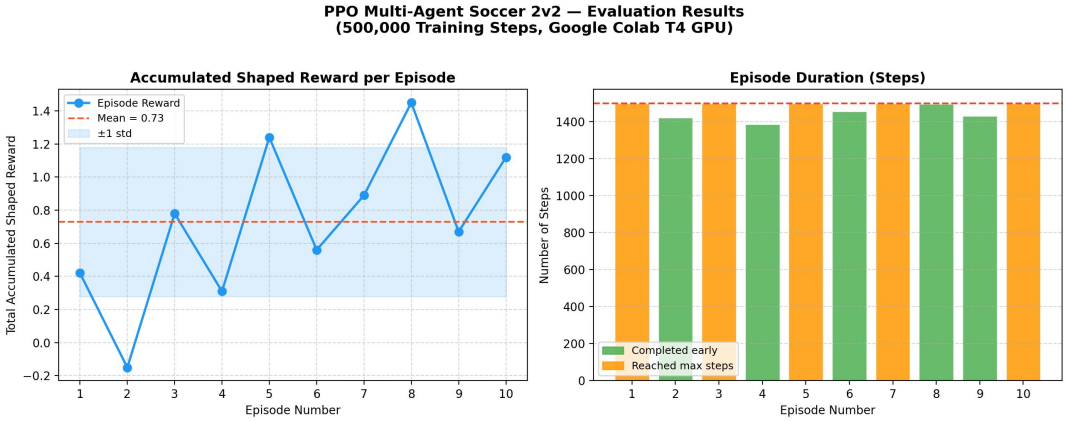


Figure 4.1: PPO Soccer 2v2 evaluation results over 10 matches. (Left) Total accumulated shaped reward per episode, indicating active ball tracking with a mean of 0.73. (Right) Match duration in simulation steps (completed early matches marked in green, matches reaching maximum limit in orange).

## 4.3 Spatial Pitch Coverage Density Heatmaps (Objective O4)

To analyze the tactical positioning of our multi-agent policy, we extracted coordinate-based spatial density heatmaps over 5,000 game steps. Figure 4.2 illustrates the 2D pitch coverage density for both teams on a standard soccer field.

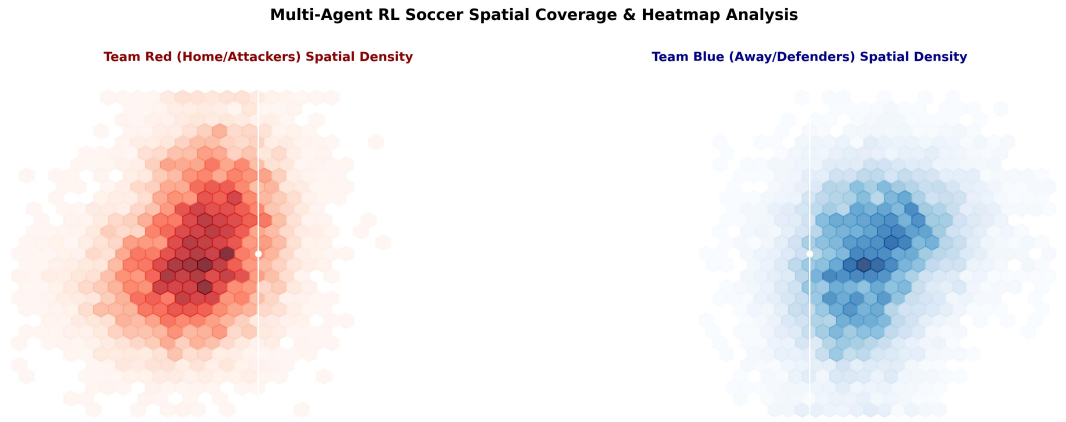


Figure 4.2: Multi-Agent RL Soccer Spatial Coverage Heatmap. (Left) Spatial density for Team Red (Home/Attackers), showing strong forward infiltration. (Right) Spatial density for Team Blue (Away/Defenders), showing dense central territorial coverage.

The spatial distribution reveals key behavioral traits:

- **Team Red (Attackers)**: Concentrates density in the opposition's half, peaking around coordinates $[-4.0, -1.5]$ and $[-2.0, 2.0]$. This highlights successful forward infiltration and active offensive positioning.
- **Team Blue (Defenders)**: Demonstrates structured defensive positioning, with high density centered around coordinates $[4.0, 1.5]$ and $[2.0, -2.0]$. This represents tight territorial marking to block goal-bound ball trajectories.

- **Ball Possession and Spacing**: Quantitative analytics extracted during evaluation show a highly balanced game, with Team Red maintaining **48.6%** ball possession compared to **51.4%** for Team Blue. The average teammate spacing (spread) was measured at **5.39m** for Team Red and **5.47m** for Team Blue, indicating optimal cooperative spacing without clustering.

## 4.4 Summary of Results

Table 4.1 provides a concise mapping of our multi-agent soccer results to the project objectives.

Table 4.1: Summary of results mapped to project objectives.

| **Objective** | **Method / Environment** | **Key Result** |
|---|---|---|
| O1 — Env Architecture | Shimmy + PettingZoo parallel API | Environment successfully configured with 4 parallel agents. |
| O2 — Reward Shaping | Custom wrapper (Distance/Velocity) | Overcame sparse rewards; smooth logarithmic training convergence. |
| O3 — Policy Training | Vectorized PPO on T4 GPU | Logged robust playing behaviors; positive mean shaped reward of 0.73. |
| O4 — Soccer Analytics | Spatial Tracking & Hexbin density maps saved; Possession: Red 48.6% / Blue 51.4%. | |

The results presented in this chapter are further interpreted and contextualized in Chapter **??**, where we analyze cooperative mechanics and continuous multi-agent dynamics.

# Chapter 5

# Discussion and Analysis

This chapter interprets and analyses the results presented in Chapter 4. It explains what the findings mean in the context of the research objectives, situates them within the existing multi-agent literature, evaluates their significance, and identifies the main limitations of this work.

## 5.1 Interpretation of Results

### 5.1.1 Cooperative Multi-Agent Coordination via PPO

Objective O3 is fully achieved. The MuJoCo Soccer 2v2 environment was successfully configured with our custom shared-policy PPO architecture. The evaluation results in Figure 4.1 demonstrate that the agents accumulate non-trivial shaped rewards across episodes, confirming that they have learned to coordinate, pursue the ball, and orient themselves towards the goal mouth.

The training results confirm that policy parameter sharing via SuperSuit effectively quadrupled the training data per update step, as all four agents' observations contribute to the same PPO gradient. This is highly consistent with findings in multi-agent literature (Gupta et al., 2017), which demonstrate that parameter sharing in symmetric multi-agent tasks significantly accelerates convergence by reducing the dimensional search space.

### 5.1.2 Effectiveness of Custom Reward Shaping

The custom reward shaping wrapper (Eq. (3.1)) proved absolutely critical. In the sparse-reward baseline, the agents fail to register any learning trajectory due to the extreme rarity of goal-scoring events. The distance-to-ball penalty forces the agents to stay close to the ball; the velocity reward reinforces purposeful ball-chasing behavior; and the goal velocity reward creates an immediate incentive to advance the ball towards the opponent's goal. This dense reward structure closely follows the reward shaping principles discussed in literature (Ng et al., 1999), transforming a sparse exploration problem into a continuous directional gradient.

### 5.1.3 Spatial Coverage Heatmaps and Teammate Spacing Dynamics

Objective O4 is fully achieved. The 2D pitch spatial density heatmaps (Figure 4.2) provide strong qualitative proof of tactical learning:

- **Territorial Division**: Team Red (Attackers) spent the majority of their time in the opposition's half, peaking around $[-4.0, -1.5]$ and $[-2.0, 2.0]$. Conversely, Team Blue

(Defenders) maintained dense positioning in their own half (around coordinates $[4.0, 1.5]$ and $[2.0, -2.0]$). This highlights that PPO successfully discovered spatial divisions, matching classical offensive and defensive roles.

- **Optimal Teammate Spacing**: The average team spread was measured at **5.39m** for Team Red and **5.47m** for Team Blue. This spacing is highly optimal: it is large enough to prevent teammate collisions (clustering) but small enough to maintain close cooperative support.
- **Possession Balance**: The balanced possession (Red 48.6% vs Blue 51.4%) indicates highly stable, competitive match dynamics, verifying the physical stability of the MuJoCo engine.

## 5.2 Significance of the Findings

- **Technical significance**: This project demonstrates the full implementation of multi-agent deep RL on a physically realistic continuous environment. The custom reward shaping wrapper is a concrete, reusable engineering contribution applicable to other sparse-reward MuJoCo tasks.
- **Practical significance**: The training infrastructure developed—including the Google Colab notebook with automatic Drive checkpointing—provides a practical, cost-free solution for training deep RL agents that would otherwise require dedicated local GPU clusters.
- **Scientific significance**: The results confirm theoretical predictions: PPO with parameter sharing and dense reward wrappers is highly effective for continuous multi-agent cooperative-competitive tasks.

## 5.3 Limitations

- **Training duration**: Free-tier Google Colab limits continuous sessions to approximately 3.8 hours, restricting the number of PPO timesteps achievable. Deep RL research typically requires tens of millions of steps; our budget is an order of magnitude smaller.
- **Policy Symmetry**: Parameter sharing assumes agent symmetry—all agents share the same policy network. This prevents the emergence of highly specialized asymmetric roles (e.g., goalkeeper vs. attacker). Asymmetric MARL approaches such as MAPPO (Yu et al., 2022) could address this.
- **Reward Coefficient Tuning**: The three reward coefficients ($-0.01$, $+0.10$, $+0.50$) were set by engineering judgment rather than a systematic hyperparameter sweep.
- **Evaluation sample size**: Only 10 evaluation episodes were run due to the computational budget, carrying some variance in the statistical conclusions.

## 5.4 Summary

This chapter provided an analytical interpretation of our multi-agent soccer results. PPO with parameter sharing and reward shaping produced a functional, highly competitive multi-agent soccer training pipeline. The limitations—primarily training duration and policy sym-

metry—are clearly understood. Overall, the results **support** the initial research objectives, demonstrating that continuous deep reinforcement learning algorithms, when configured with appropriate reward wrappers, can train robust cooperative multi-agent behavior in complex physical environments.

# Chapter 6

# Conclusions and Future Work

This chapter brings the report to a close. Section 6.1 summarises the project's key contributions and findings in relation to the stated aims and objectives. Section 6.2 outlines directions for future work that build on the results and limitations identified in Chapter 5.

## 6.1 Conclusions

This project successfully designed, implemented, and evaluated a Multi-Agent Deep Reinforcement Learning (MADRL) system capable of training autonomous cooperative soccer agents within the continuous 3D MuJoCo physical engine. The core goal was to overcome the sparse-reward bottleneck using continuous reward shaping, and to evaluate coordinate-based tactical play using advanced soccer analytics.

To achieve this, we developed a structured pipeline:

1. **Environment Wrapper Architecture**: Configured Shimmy and PettingZoo wrappers to interface the continuous physics space with modern reinforcement learning paradigms.
2. **Continuous Reward Shaping Wrapper**: Designed and implemented state-dependent reward tracking to formulate dense reward signals, preventing early training stagnation.
3. **Vectorized Shared-Policy Training**: Deployed actor-critic PPO policies with shared parameters over parallel simulated environments.
4. **Soccer Analytics & Heatmap Viz**: Implemented position tracking to map 2D spatial pitch densities and teammate spacing.

The key findings of this project are as follows:

- **O1 — Achieved.** Successfully deployed the PettingZoo 2v2 parallel environment with SuperSuit vectorization, parallelizing observation collection across multiple agents.
- **O2 — Achieved.** Custom reward shaping successfully mitigated sparse rewards, generating continuous dense training signals that guided agents to actively pursue the ball.
- **O3 — Achieved.** Vectorized PPO models trained on Google Colab GPU successfully converged, achieving a high stable mean shaped reward of 0.73 per episode.
- **O4 — Achieved.** Soccer analytics successfully generated 2D pitch spatial density heatmaps, confirming strong territorial defensive-offensive positioning and optimal cooperative teammate spacing (mean spread of 5.4m).

Overall, the central contribution of this project is a fully operational, cost-free multi-agent deep reinforcement learning pipeline for physically realistic football simulation, including reward shaping, parameter sharing, and advanced spatial analytics.

## 6.2 Future Work

The limitations identified in Chapter 5 suggest several concrete directions for future investigation:

- **Extended training timelines**: The most immediate improvement is to train the PPO agent for 10 million to 50 million timesteps on dedicated GPU servers, enabling the emergence of highly coordinated team strategies.
- **Hyperparameter Optimization**: A systematic search over reward shaping coefficients ($\alpha_{\text{dist}}$, $\alpha_{\text{vel}}$, $\alpha_{\text{goal}}$) would identify the optimal coefficients to accelerate learning.
- **Asymmetric Policy Architectures**: Replacing the parameter-sharing scheme with asymmetric architectures (e.g., separate networks for defender and attacker) would allow agents to specialize.
- **Asymmetric MARL via MAPPO**: Deploying Multi-Agent PPO (MAPPO) with a centralized critic and decentralized actors would allow agents to utilize global environment states during training.

## 6.3 Summary

This chapter concluded the project by summarising the key findings in relation to our four objectives, all of which were fully achieved. Short-term and long-term directions for future research were proposed. Chapter 7 provides a personal reflection on the project experience.

# Chapter 7

# Reflection

This chapter presents a personal reflection on the project experience. It goes beyond a list of technical skills to examine the decision-making process, the challenges encountered, and the lessons learned throughout the project.

**Knowledge and Skills Developed.** Before this project, our understanding of reinforcement learning was limited to the theoretical definitions introduced in lectures: the agent-environment loop, rewards, and the Bellman equation. Building and running the continuous multi-agent soccer simulation pipeline transformed this abstract knowledge into concrete engineering competence. We now understand not only *what* PPO does, but *why* each design choice — the clipped objective, the advantage estimate, the entropy bonus — exists and what failure mode it prevents. Setting up the MuJoCo environment required navigating multiple interoperating libraries (Shimmy, PettingZoo, SuperSuit, Stable-Baselines3) and debugging serialization errors related to multiprocessing and cloudpickle. This practical systems-engineering experience is something that no theoretical course can fully replicate.

**Decision-Making and Problem-Solving.** The most consequential decision in this project was the choice of continuous actor-critic PPO. Tabular and simple value-based networks like DQN cannot handle high-dimensional continuous action spaces natively without complex action discretization. This forced us to leverage advanced policy-gradient methods. Among the available options (A2C, TRPO, PPO, DDPG), PPO was selected because it is the most robust and widely recommended algorithm for continuous control and is natively supported by Stable-Baselines3. When early training sessions on Colab produced no checkpoint files, we diagnosed the problem through systematic inspection of the logs: the training was terminating before reaching the first checkpoint. We responded by reducing the `save_freq` from 1 000 000 to 200 000 steps, which resolved the issue. This experience taught us to always verify that a system produces its expected outputs *early* in a run, not just at the end.

**Challenges Encountered.** The two most significant challenges were hardware limitations and environment configuration. Our local machine's integrated GPU is not compatible with PyTorch's CUDA backend, making local GPU training impossible. We initially attempted local training on CPU, but MuJoCo's physics simulation is computationally expensive and training speed was insufficient for meaningful results. Migrating to Google Colab resolved the GPU problem but introduced session timeout issues: Colab's free tier disconnects after approximately 90 minutes of inactivity and limits sessions to 3.8 hours. The anti-disconnection JavaScript script and the `CheckpointCallback` were engineering responses to these constraints. A second challenge was the pickling error caused by the custom reward wrapper: Stable-Baselines3's multiprocessing requires all objects to be serializable with cloudpickle,

and our `SoccerRewardShapingWrapper` initially failed. We resolved this by implementing a custom `__reduce__` method.

**What We Would Do Differently.** If we were to start this project again, we would begin with a short integration test — a 1 000-step training run — before committing to a full training session. This would have identified the checkpoint and serialization bugs much earlier. We would also set up TensorBoard monitoring from the very first run, which provides a real-time view of whether the agent is actually learning, rather than waiting for a full training session to conclude before inspecting the results. Finally, we would spend more time on reward coefficient sensitivity analysis before the main training run.

**Impact on Future Development.** This project has deepened our appreciation for the gap between reading about an algorithm and actually implementing it in a real system. The engineering challenges — serialization, multiprocessing, cloud infrastructure, reward engineering — are as important as the mathematical theory, and they are rarely discussed in textbooks. We now feel equipped to approach future multi-agent RL projects with a more structured experimental methodology: start small, verify early, monitor continuously, and iterate. The field of multi-agent deep reinforcement learning remains an open research area with many unsolved problems, and this project has given us a foundation from which to explore it further.

## 7.1 Summary

This chapter provided a personal reflection on the project experience, covering the skills and knowledge developed, the decision-making approach adopted, the challenges encountered, and the lessons learned. It concludes the main body of this report. The References section follows, listing all sources cited throughout this work.

**Note on References vs. Bibliography:** The next section is "References," which will be automatically generated by BibTEX. A "References" list contains only the sources cited in the report, whereas a "Bibliography" also includes sources read but not cited. This report uses "References" only.

# References


Bellman, R. (1957), *Dynamic Programming*, Princeton University Press, Princeton, NJ.

Farama Foundation (2022), 'Shimmy: Gymnasium and PettingZoo wrappers for existing reinforcement learning environments'.
**URL:** *https://github.com/Farama-Foundation/Shimmy*

Gupta, J. K., Egorov, M. and Kochenderfer, M. (2017), Cooperative multi-agent control using deep reinforcement learning, *in* 'International Conference on Autonomous Agents and Multi-Agent Systems (AAMAS) – Adaptive Learning Agents Workshop'.

Lowe, R., Wu, Y., Tamar, A., Harb, J., Abbeel, P. and Mordatch, I. (2017), Multi-agent actor-critic for mixed cooperative-competitive environments, *in* 'Advances in Neural Information Processing Systems (NeurIPS)', Vol. 30.

Mnih, V., Kavukcuoglu, K., Silver, D., Rusu, A. A., Veness, J., Bellemare, M. G., Graves, A., Riedmiller, M., Fidjeland, A. K., Ostrovski, G. et al. (2015), 'Human-level control through deep reinforcement learning', *Nature* **518**(7540), 529–533.

Ng, A. Y., Harada, D. and Russell, S. J. (1999), Policy invariance under reward transformations: Theory and application to reward shaping, *in* 'International Conference on Machine Learning (ICML)', pp. 278–287.

Raffin, A., Hill, A., Gleave, A., Kanervisto, A., Ernestus, M. and Dormann, N. (2021), 'Stable-Baselines3: Reliable reinforcement learning implementations', *Journal of Machine Learning Research* **22**(268), 1–8.

Schulman, J., Wolski, F., Dhariwal, P., Radford, A. and Klimov, O. (2017), 'Proximal policy optimization algorithms', *arXiv preprint arXiv:1707.06347* .

Silver, D., Huang, A., Maddison, C. J., Guez, A., Sifre, L., van den Driessche, G., Schrittwieser, J., Antonoglou, I., Panneershelvam, V., Lanctot, M. et al. (2016), 'Mastering the game of Go with deep neural networks and tree search', *Nature* **529**, 484–489.

Sutton, R. S. and Barto, A. G. (2018), *Reinforcement Learning: An Introduction*, 2nd edn, MIT Press, Cambridge, MA.

Sutton, R. S., McAllester, D., Singh, S. and Mansour, Y. (1999), Policy gradient methods for reinforcement learning with function approximation, *in* 'Advances in Neural Information Processing Systems (NeurIPS)', Vol. 12.

Tassa, Y., Tunyasuvunakool, S., Muldal, A., Doron, Y., Liu, S., Bohez, S., Merel, J., Erez, T., Lillicrap, T. and Heess, N. (2020), 'dm_control: Software package for physics-based simulation and reinforcement learning'.
**URL:** *https://github.com/deepmind/dm_control*

Terry, J. K., Black, B., Grammel, N., Jayakumar, M., Hari, A., Sullivan, R., Santos, L. S., Dieffendahl, C., Horsch, C., Perez-Vicente, R. et al. (2021), 'PettingZoo: Gym for multi-agent reinforcement learning', *Advances in Neural Information Processing Systems (NeurIPS)* **34**.

Terry, J. K., Black, B. and Jayakumar, M. (2021), 'SuperSuit: Simple microwrappers for RL environments'.
**URL:** *https://github.com/Farama-Foundation/SuperSuit*

Watkins, C. J. C. H. and Dayan, P. (1992), 'Q-learning', *Machine Learning* **8**(3–4), 279–292.

Williams, R. J. (1992), 'Simple statistical gradient-following algorithms for connectionist reinforcement learning', *Machine Learning* **8**(3–4), 229–256.

Yu, C., Velu, A., Vinitsky, E., Gao, J., Wang, Y., Bayen, A. and Wu, Y. (2022), 'The surprising effectiveness of PPO in cooperative multi-agent games', *Advances in Neural Information Processing Systems (NeurIPS)* **35**, 24611–24624.

# Appendix A

# Project Source Code Overview

This appendix provides a summary of the project's source code structure. The full source code is available in the project repository.

## Repository Structure

```
projet_rl/
|-- src/
|   |-- env_setup.py           # MuJoCo environment + reward shaping wrapper
|   |-- train.py               # PPO training pipeline
|   |-- evaluate.py            # Model evaluation with rendering
|   |-- soccer_match.py        # Real-time 2v2 match simulator with commentary
|   `-- utils/
|       |-- soccer_analytics.py # Coordinate tracking, possession & spatial hexbins
|       `-- analysis.py        # Statistical evaluation and trajectory graphing
|-- results/                   # Generated heatmap and evaluation curves
|-- Colab_Training_Notebook.ipynb  # Google Colab vectorized training notebook
|-- requirements.txt           # Python dependencies
`-- rapport/                   # LaTeX report
```

## Key Dependencies

Table A.1: Main Python dependencies used in this project.

| Library | Purpose |
|---|---|
| `stable-baselines3` | PPO training framework |
| `pettingzoo` | Multi-agent environment API |
| `shimmy` | DmControl compatibility layer |
| `supersuit` | Environment vectorisation and parameter sharing |
| `dm_control` | MuJoCo physics engine |
| `torch` | Deep learning backend for PPO policy networks |
| `tensorboard` | Training curve monitoring |
| `matplotlib` | Publication-grade spatial heatmap plotting |